

CSCI-4208: Developing Advanced Web Applications

Week 10: Lecture 16

Node - Core Modules

(i.e. Standard Library)

Node.js | Modules

In Node.js, **Modules** are the blocks of encapsulated code that communicates with an external application on the basis of their related functionality. Modules can be a single file or a collection of multiples files/folders. The reason programmers are heavily reliant on modules is because of their re-usability as well as the ability to break down a complex piece of code into manageable chunks.

Modules are of three types:

- Core Modules
- local Modules
- Third-party Modules

Core Modules:

Node.js has many built-in modules that are part of the platform and comes with Node.js installation. These modules can be loaded into the program by using the **require** function.

Syntax:

```
var module = require('module_name');
```

The `require()` function will return a JavaScript type depending on what the particular module returns. The following example demonstrates

Node - core modules: Full Listing

Name	Description	Name	Description
assert	provides a set of assertion functions useful for testing	process	provides information about, and control over, the current Node.js process
buffer	provides the ability to handle buffers containing binary data	querystring	provides utilities for parsing and formatting URL query strings
child_process	provides the ability to spawn child processes	readline	provides an interface for reading data from a Readable stream
console	provides a simple debugging console	repl	provides a Read-Eval-Print-Loop (REPL) implementation
cluster	splits a process into multiple workers to benefit from multi-core systems	stream	an abstract interface for working with streaming data
crypto	provides cryptographic functionality	string_decoder	provides an API for decoding Buffer objects into strings
dgram	provides an implementation of UDP Datagram sockets	timers	provide functions to schedule callbacks for some future time
dns	provides name resolution and DNS lookups	tls	provides implementation for Transport Layer Security & Secure Socket Layer
events	provides an API for managing events	tty	provides functionality used to perform I/O operations in a text terminal
fs	provides an API for interacting with the file system	url	provides utilities for URL resolution and parsing
http	provides an HTTP client/server implementation	util	supports the Node.js internal APIs, useful for app and module developers
http2	provides an HTTP/2 client/server implementation	v8	exposes APIs that are specific to the version of V8 built into the Node.js binary
https	provides an HTTPS client/server implementation	vm	enables compiling and running code within V8 Virtual Machine contexts
net	provides an asynchronous network API	wasi	provides an implementation of the WebAssembly System Interface specification
os	provides operating system-related utility methods and properties	worker	enables the use of threads that execute JavaScript in parallel
path	provides utilities for working with file and directory paths	zlib	provides compression functionality
perf_hooks	to enable the collection of performance metrics		

Node - events module

The events module provides us the EventEmitter class, which is key to working with events in Node.

```
const EventEmitter = require('events');  
const emitter = new EventEmitter();
```

Here's a detailed description of the most useful methods:

emitter.addListener()	Alias for emitter.on().
emitter.emit()	Emits an event. It synchronously calls every event listener in the order they were registered.
emitter.eventNames()	Return an array of strings that represent the events registered on the current EventEmitter:
emitter.getMaxListeners()	Get the maximum amount of listeners one can add to an EventEmitter object, which defaults to 10 but can be changed with setMaxListeners()
emitter.listenerCount()	Get the count of listeners of the event passed as parameter
emitter.listeners()	Gets an array of listeners of the event passed as parameter
emitter.off()	Alias for emitter.removeListener()
emitter.on()	Adds a callback function that's called when an event is emitted.
emitter.once()	Adds a callback for when an event is emitted for the first time after registering this. This callback is only going to be called once, never again.
emitter.prependListener()	When you add a listener using on or addListener, it's added last in the queue of listeners, and called last. Using prependListener it's added, and called, before other listeners.
emitter.prependOnceListener()	When you add a listener using once, it's added last in the queue of listeners, and called last. Using prependOnceListener it's added, and called, before other listeners.
emitter.removeAllListeners()	Removes all listeners of an event emitter object listening to a specific event:
emitter.removeListener()	Remove a specific listener. You can do this by saving the callback function to a variable, when added, so you can reference it later
emitter.setMaxListeners()	Sets the maximum amount of listeners one can add to an EventEmitter object, which defaults to 10 but can be increased or lowered.

Node - fs module

The fs module provides a lot of very useful functionality to access and interact with the file system.

```
const fs = require('fs')
```

Here's a detailed description of the most useful methods:

fs.access()	check if the file exists and Node can access it
fs.appendFile()	append data to a file. If the file does not exist, it's created
fs.chmod()	change the permissions of a file by the filename passed.
fs.chown()	change the owner & group of a file by the filename passed.
fs.close()	close a file descriptor
fs.copyFile()	copies a file
fs.createReadStream()	create a readable file stream
fs.createWriteStream()	create a writable file stream
fs.link()	create a new hard link to a file.
fs.mkdir()	create a new folder
fs.mkdtemp()	create a temporary directory
fs.open()	set the file mode
fs.readdir()	read the contents of a directory

fs.readFile()	read the content of a file. Related: fs.read()
fs.readlink()	read the value of a symbolic link
fs.realpath()	resolve relative file path pointers (., ..) to the full path
fs.rename()	rename a file or folder
fs.rmdir()	remove a folder
fs.stat()	returns the status of the file identified by the filename passed.
fs.symlink()	create a new symbolic link to a file
fs.truncate()	truncate a file to a specified length by the filename passed.
fs.unlink()	remove a file or a symbolic link
fs.unwatchFile()	stop watching for changes on a file
fs.utimes()	change the timestamp of a file by the filename passed.
fs.watchFile()	start watching for changes on a file.
fs.writeFile()	write data to a file.

Node - fs module

One peculiar thing about the `fs` module is that all the methods are asynchronous by default, but they can also work synchronously by appending `Sync`.

For example:

- `fs.rename()`
- `fs.renameSync()`
- `fs.write()`
- `fs.writeSync()`

This makes a huge difference in your application flow.

Node - fs module

For example let's examine the `fs.rename()` method. The asynchronous API is used with a callback:

```
const fs = require('fs')

fs.rename('before.json', 'after.json', (err) => {
  if (err) {
    return console.error(err)
  }

  //done
})
```

A synchronous API can be used like this, with a try/catch block to handle errors:

```
const fs = require('fs')

try {
  fs.renameSync('before.json', 'after.json')
  //done
} catch (err) {
  console.error(err)
}
```

The key difference here is that the execution of your script will block in the second example, until the file operation succeeded.

Node - os module

This module provides many functions that you can use to retrieve information from the underlying operating system and the computer the program runs on, and interact with it.

```
const os = require('os')
```

Here's a detailed description of the most useful methods:

os.arch()	Return the string that identifies the underlying architecture, like arm, x64, arm64.
os.cpus()	Return information on the CPUs available on your system.
os.endianness()	Return BE or LE depending if Node was compiled with Big Endian or Little Endian.
os.freemem()	Return the number of bytes that represent the free memory in the system.
os.homedir()	Return the path to the home directory of the current user.
os.hostname()	Return the hostname.
os.loadavg()	Return the calculation made by the operating system on the load average. It only returns a meaningful value on Linux and macOS.
os.networkInterfaces()	Returns the details of the network interfaces available on your system.
os.platform()	Return the platform that Node was compiled for
os.release()	Returns a string that identifies the operating system release number
os.tmpdir()	Returns the path to the assigned temp folder.
os.totalmem()	Returns the number of bytes that represent the total memory available in the system.
os.type()	Identifies the operating system:
os.uptime()	Returns the number of seconds the computer has been running since it was last rebooted.
os.userInfo()	Returns information about the current user

Node - path module

The path module provides a lot of very useful functionality to access and interact with the file system.

```
const path = require('path');
```

This module provides `path.sep` which provides the path segment separator (\ on Windows, and / on Linux / macOS), and `path.delimiter` which provides the path delimiter (; on Windows, and : on Linux / macOS).

Here's a detailed description of the most useful methods:

<code>path.basename()</code>	Return the last portion of a path. A second parameter can filter out the file extension:	<i>Ex: <code>path.basename('home/public/index.html')</code></i>
<code>path.dirname()</code>	Return the directory part of a path:	<i>Ex: <code>path.dirname('home/public/index.html')</code></i>
<code>path.extname()</code>	Return the extension part of a path	<i>Ex: <code>path.extname('home.html')</code></i>
<code>path.isAbsolute()</code>	Returns true if it's an absolute path	<i>Ex: <code>path.isAbsolute('/home.html')</code></i>
<code>path.join()</code>	Joins two or more parts of a path:	<i>Ex: <code>path.join('home', 'index.html')</code></i>
<code>path.normalize()</code>	Tries to calculate the actual path when it contains relative specifiers like . or .., or double slashes:	<i>Ex: <code>path.normalize('./home/index.html')</code></i>
<code>path.parse()</code>	Parses a path to an object with the segments that compose it: • root: the root • dir: the folder path starting from the root • base: the file name + extension • name: the file name • ext: the file extension	<i>Ex: <code>path.parse('./user/Desktop/index.html')</code></i>
<code>path.relative()</code>	Accepts 2 paths as args. Returns the relative path from 1st path to 2nd, based on current working directory	<i>Ex: <code>path.relative('app/public', 'index.html')</code></i>
<code>path.resolve()</code>	You can get the absolute path calculation of a relative path using <code>path.resolve()</code>	<i>Ex: <code>path.resolve('Desktop')</code></i>

Node - url module

The url module provides a lot of very useful functionality to access and interact with the file system.

```
const url = require('url');
```

Here's a detailed description of the most useful methods:

<code>url.URL()</code>	Creates a new URL object by parsing the input relative to the base. <i>Ex: <code>new url.URL('/foo', 'https://example.org')</code></i>
<code>url.URLSearchParams()</code>	Parse the string as a query string, and use it to instantiate a new URLSearchParams object. <i>Ex: <code>new url.URLSearchParams('?name=val&age=10')</code></i>
<code>url.format()</code>	Returns a customizable serialization of a URL String representation <i>Ex: <code>url.format('hello world.html')</code></i>
<code>url.parse()</code>	The <code>url.parse()</code> method takes a URL string, parses it, and returns a URL object.
<code>url.resolve()</code>	The <code>url.resolve()</code> method resolves a target URL relative to a base URL in a manner similar to that of a Web browser resolving an anchor tag HREF.
<code>url.urlToHttpOption()</code>	This utility function converts a URL object into an ordinary options object as expected by the <code>http.request()</code> and <code>https.request()</code> APIs.

Node - url module: URL object

The url module provides a lot of very useful functionality to access and interact with the file system.

```
const url = require('url');  
const myUrl = new url.URL('https://example.org:8080/home/index.html?name=foo#idSelector');
```

Here's a detailed description of the most useful URL properties:

myUrl.href	Gets and sets the serialized URL. Getting the value of the href property is equivalent to calling url.toString().
myUrl.origin	Gets the read-only serialization of the URL's origin.
myUrl.protocol	Gets and sets the protocol portion of the URL.
myUrl.host	Gets and sets the host portion of the URL.
myUrl.hostname	Gets and sets the host name portion of the URL. The key difference between url.host and url.hostname is that url.hostname does not include the port.
myUrl.port	Gets and sets the port portion of the URL.
myUrl.pathname	Gets and sets the path portion of the URL.
myUrl.search	Gets and sets the serialized query portion of the URL.
url.searchParams	Gets the URLSearchParams object representing the query parameters of the URL.
url.hash	Gets and sets the fragment portion of the URL.

Node - url module: URLSearchParams object

The url module provides a lot of very useful functionality to access and interact with the file system.

```
const url = require('url');  
const search = new url.URLSearchParams('?name=val&age=10');
```

Here's a detailed description of the most useful URL properties:

search.append	tbd
search.delete	tbd
search.forEach	tbd
search.getAll	tbd
search.keys	tbd
search.sort	tbd
search.values	tbd
search.entries	tbd
search.get	tbd
search.has	tbd
search.set	tbd
search.toString	tbd

Node - http module

The HTTP core module is a key module to Node networking.

- Properties

- `http.METHODS`
- `http.STATUS_CODES`
- `http.globalAgent`

- Methods

- `http.createServer()`
- `http.request()`
- `http.get()`

- Classes

- `http.Agent`
- `http.ClientRequest`
- `http.Server`
- `http.ServerResponse`
- `http.IncomingMessage`

Node - http module

It can be included using

```
const http = require('http')
```

The module provides some properties and methods, and some classes.

Node - http module

It can be included using

```
const http = require('http')
```

The module provides some properties and methods, and some classes.

Node - http module: Properties

- `http.METHODS`:
 - This property lists all the HTTP methods supported:
- `http.STATUS_CODES`:
 - This property lists all the HTTP status codes and their description
- `http.globalAgent`:
 - Points to the global instance of the Agent object, which is an instance of the `http.Agent` class. It's used to manage connections persistence and reuse for HTTP clients, and it's a key component of Node HTTP networking.

Node - http module: Methods

- `http.createServer()`
 - Return a new instance of the `http.Server` class.
- `http.request()`
 - Makes an HTTP request to a server, creating an instance of the `http.ClientRequest` class.
- `http.get()`
 - Similar to `http.request()`, but automatically sets the HTTP method to GET, and calls `req.end()` automatically.

Node - http module: Classes - http.Agent

Node creates a global instance of the `http.Agent` class to manage connections persistence and reuse for HTTP clients, a key component of Node HTTP networking.

This object makes sure that every request made to a server is queued and a single socket is reused.

It also maintains a pool of sockets. This is key for performance reasons.

Node - http module: Classes - http.ClientRequest

An `http.ClientRequest` object is created when `http.request()` or `http.get()` is called.

When a response is received, the response event is called with the response, with an `http.IncomingMessage` instance as argument.

The returned data of a response can be read in 2 ways:

- you can call the `response.read()` method
- in the response event handler you can setup an event listener for the data event, so you can listen for the data streamed into.

Node - http module: Classes - http.Server

This class is commonly instantiated and returned when creating a new server using `http.createServer()`.

Once you have a server object, you have access to its methods:

- `close()` stops the server from accepting new connections
- `listen()` starts the HTTP server and listens for connections

Node - http module: Classes - http.ServerResponse

Created by an `http.Server` and passed as the second parameter to the request event it fires.

Commonly known and used in code as `res`:

```
const server = http.createServer((req, res) => {  
  //res is an http.ServerResponse object  
})
```

The method you'll always call in the handler is `end()`, which closes the response, the message is complete and the server can send it to the client. It must be called on each response.

These methods are used to interact with HTTP headers:

- `getHeaderNames()` get the list of the names of the HTTP headers already set
- `getHeaders()` get a copy of the HTTP headers already set
- `setHeader('headername', value)` sets an HTTP header value
- `getHeader('headername')` gets an HTTP header already set
- `removeHeader('headername')` removes an HTTP header already set
- `hasHeader('headername')` return true if the response has that header set
- `headersSent()` return true if the headers have already been sent to the client

Node - http module: Classes - http.IncomingMessage

An `http.IncomingMessage` object is created by:

- `http.Server` when listening to the request event
- `http.ClientRequest` when listening to the response event

It can be used to access the response:

- status using its `statusCode` and `statusMessage` methods
- headers using its `headers` method or `rawHeaders`
- HTTP method using its `method` method
- HTTP version using the `httpVersion` method
- URL using the `url` method
- underlying socket using the `socket` method

The data is accessed using streams, since `http.IncomingMessage` implements the `Readable Stream` interface.

Node - child_process module

The child_process module provides the ability to spawn subprocesses in a manner that is similar, but not identical, to `popen(3)`

```
const { exec } = require('child_process')
```

Here's an example of using child_process to invoke command line call to bash from node:

```
const { exec } = require('child_process');

exec('"/path/to/test file/test.sh" arg1 arg2');
// Double quotes are used so that the space in the path is not interpreted as
// a delimiter of multiple arguments.

exec('echo "The \\$HOME variable is $HOME"');
// The $HOME variable is escaped in the first instance, but not in the second.
```


The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

Web Server in Node & npm

Homework

TBD